Enterprise IT Support Agentic RAG Copilot

FDE Project - Problem Statement, Proposed Solution, and Easy-to-Understand Examples

Customer:	Environment:	Use Case:
NovaRetail (fictional)	3,000 employees	Internal IT Support

1. Problem Statement

NovaRetail has a large internal IT knowledge base containing VPN instructions, password rules, MFA policies, software-installation rules, laptop troubleshooting guides, and service-desk runbooks. Even though the information already exists, employees still create repetitive support tickets because they do not know which document contains the correct answer.

A traditional keyword search is often frustrating because it can return many documents instead of one clear answer. A normal chatbot creates another risk: it may answer confidently even when it does not have reliable company information. In addition, some questions depend on current public information that may not yet exist in the company's private knowledge base.

The core business challenge

> **Employees need fast answers, but the company needs those answers to be grounded, trustworthy, transparent, and based on private company knowledge whenever possible.**

2. Why Simple RAG Is Not Enough

Simple RAG	Problem in the real world
Question -> Retrieve -> Generate	Retrieved chunks may be weak or unrelated, but the model may still generate an answer.
Uses only the private KB	It cannot answer well when the required information is missing or outdated.
No evidence decision	The application does not decide whether the retrieved information is good enough.

3. Proposed Solution

Build an **Enterprise IT Support Agentic RAG Copilot**. The system searches the trusted private company knowledge base first. It then evaluates the retrieved evidence before generating an answer. If the private evidence is weak, the workflow can use Tavily web search. If the evidence is still weak, the system rewrites the query and retries instead of blindly answering.

1	Employee asks a question
2	LangGraph routes the request
3	Retrieve relevant chunks from the private Pinecone knowledge base
4	Grade whether the private evidence is strong enough
5A	GOOD -> Generate a grounded answer from company knowledge
5B	WEAK -> Search the web using Tavily
6	Grade the web evidence
7	If needed, rewrite the query and retry
8	Return the answer, sources, and decision trace

4. Easy Example #1 - Answer Found in the Company KB

Employee: "How do I connect to the company VPN from home?"

The VPN procedure already exists in the private IT handbook. The system retrieves the relevant chunks from Pinecone, grades them as useful evidence, and generates the answer from the internal knowledge base. There is no reason to search the public web.

Easy explanation: The AI first checks the company's own trusted documents. Because it finds the answer there, it stops and answers from those documents.

5. Easy Example #2 - Private Knowledge Is Not Enough

Employee: "What is the latest Microsoft Teams outage guidance?"

The private IT documents may not contain current outage information. The system retrieves private knowledge first, but the evidence grader identifies that the information is insufficient. LangGraph then routes the workflow to Tavily web search. The web evidence is graded before the final answer is produced.

Easy explanation: The AI does not pretend that the company documents contain the answer. It recognizes the gap, searches an external source, checks that evidence, and then responds.

6. What Makes This Agentic RAG?

Normal RAG follows a mostly fixed path: **Question -> Retrieve -> Generate**. This project makes decisions during execution. It can route, retrieve, evaluate evidence, choose between private knowledge and web search, rewrite a weak query, retry, and then generate a grounded answer.

Normal RAG	Agentic RAG in this project
Retrieve once	Retrieve, grade, and retry when necessary
Fixed flow	Conditional LangGraph routing
Generate after retrieval	Generate only after evidence evaluation
Private KB only	Private KB first, web fallback when needed

7. Proposed Product Components

• **LangGraph:** controls the agentic decision workflow.

• **Pinecone:** stores the company's private embedded knowledge.

• **HuggingFace embeddings:** convert document chunks and queries into vectors.

• **Groq LLM:** performs routing, grading, query rewriting, and grounded generation.

• **Tavily:** provides external web search when internal evidence is insufficient.

• **FastAPI:** exposes the application through backend APIs.

• **HTML/CSS/JavaScript UI:** gives employees a simple chat experience and displays the execution trace.

• **SQLite audit logging:** records the decision path for basic debugging and transparency.

• **Document ingestion:** lets authorized staff add PDF, TXT, Markdown, and DOCX knowledge.

8. Expected Business Value

• Reduce repetitive IT support tickets.

• Help employees find answers faster.

• Use private company knowledge before public information.

• Reduce unsupported or hallucinated answers by grading evidence.

• Handle knowledge gaps through controlled web fallback.

• Make AI decisions easier to inspect through source and workflow traces.

• Allow the knowledge base to grow as authorized staff add new documents.

9. FDE Perspective

The goal of the Forward Deployed Engineer is not simply to demonstrate RAG in a notebook. The FDE starts from the customer's support problem and delivers a usable solution: knowledge ingestion, agentic

workflow, APIs, user interface, auditability, testing, packaging, and a path toward deployment.

> **Simple takeaway:**
> Customer Problem -> Understand the Workflow -> Connect Company Knowledge -> Build Agentic RAG -> Expose APIs -> Build the UI -> Test -> Deploy -> Observe -> Improve

Project basis: FDE_Agentic_RAG_IT_Copilot supplied for this lesson. NovaRetail is a fictional company used to make the scenario easy to teach.